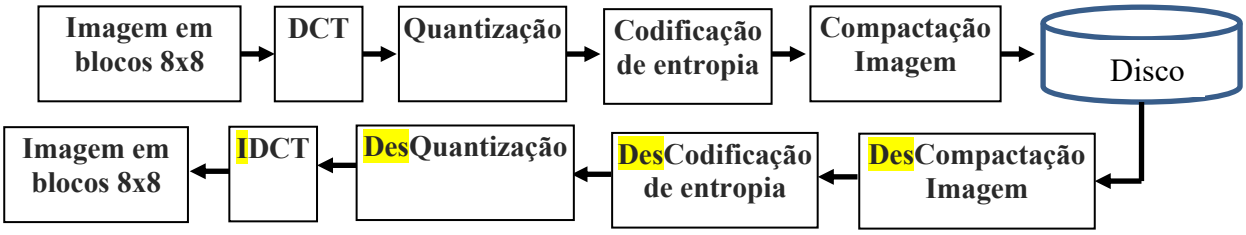
Universidade Federal de Santa Maria
Departamento de Computação Aplicada
Prof. Cesar Tadeu Pozzer
Disciplina: Computação Gráfica Avançada
04/07/2026
pozzer@inf.ufsm.br

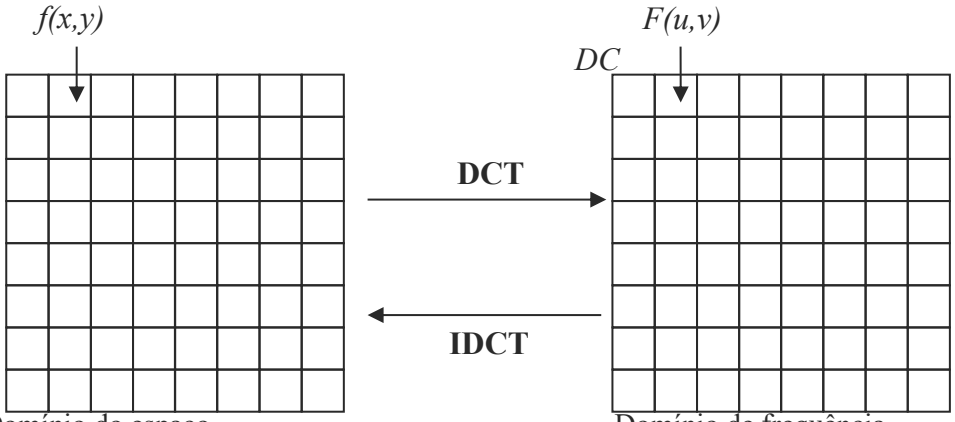
Formato JPEG

- Formato de arquivo de imagem que tem como principais características apresentar [1]:
  - Compactação da imagem
  - Compressão da imagem variável
  - Modo progressivo
  - Modo Hierárquico: em diferentes resoluções (*Mipmap*)
  - 16 M cores
  - Faz uso da DCT (*Discrete Cosine Transform*) para comprimir a imagem



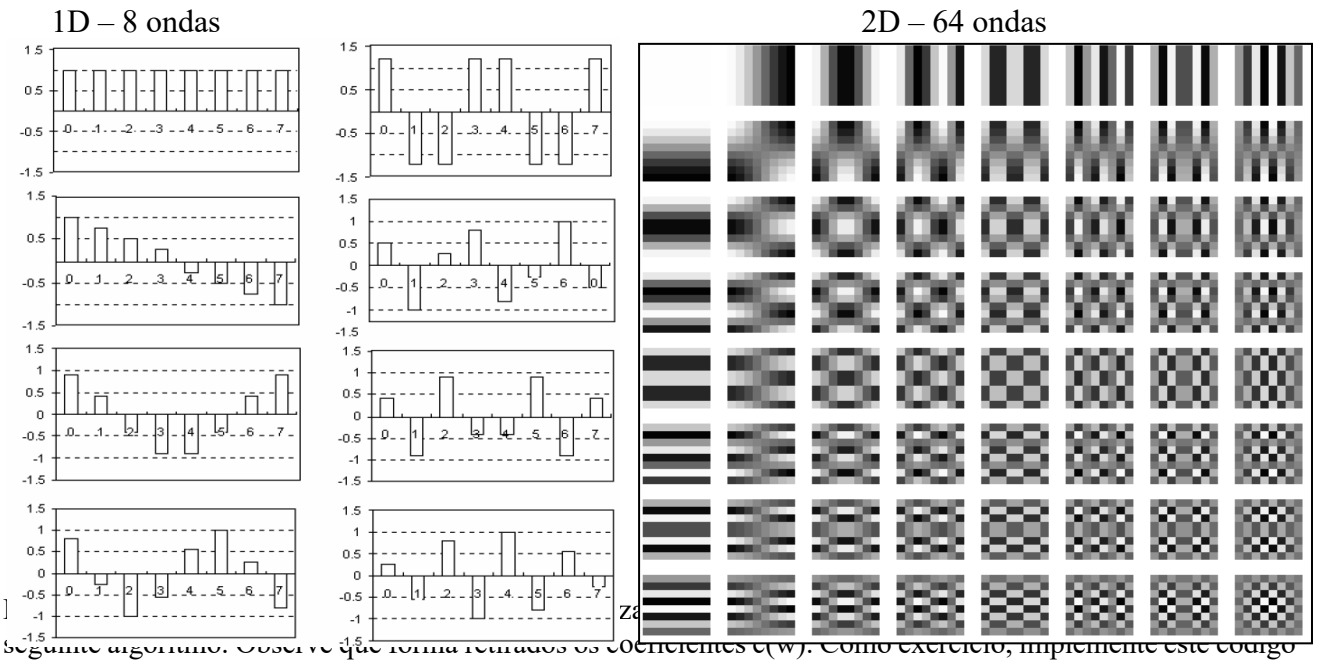
Transformadas

- Tem a função de transformar sinais do domínio tempo-espaço para o domínio da frequência. Diversos processamentos em sinais podem ser melhor realizados no espectro da frequência, como no caso da compressão de imagens;
  - Uma imagem é um sinal no domínio do espaço
  - Áudio é um sinal no domínio do tempo
- A DCT é similar a transformada discreta de Fourier: ela transforma um sinal ou imagem (1D ou 2D) do domínio espacial para o domínio da frequência.
- Uma imagem no domínio da frequência é caracterizada pela variação de tom entre pixels vizinhos. Quanto maior for a variação de intensidade, maior é a frequência da imagem. Imagens que caracterizam um texto escaneado, por exemplo, são marcadas por altas frequências, visto que existem transições frequentes do branco (cor do papel) para o preto (cor da letra);
- A DCT tem a característica de concentrar a maior parte da energia em seus coeficientes iniciais, ou seja, de baixas frequências;
- Imagens contêm uma grande parcela de redundância, ou seja, é grande a correlação entre os pixels formadores da imagem. Os processos de codificação por transformada reduzem a correlação. Técnicas que aplicam transformada geralmente dividem a imagem em blocos para diminuir a complexidade do algoritmo. A imagem é dividida em blocos de tamanho fixo, geralmente 8x8, 16x16, etc. Cada bloco é tratado como um vetor e codificado com a transformada, independentemente dos outros blocos. Posteriormente, para fazer a decodificação, este vetor é processado com a operação inversa da transformada, produzindo o vetor reconstruído. Os blocos reconstruídos são reunidos novamente para formar a imagem [9];
- A ideia é encontrar N valores (**coeficientes**) e N funções (**formas de onda ortogonais**) que, quando multiplicados e somados, podem reconstruir qualquer informação (representar quaisquer N amostras).



- Se a imagem for colorida (R,G,B), deve-se aplicar a DCT para cada componente da imagem. Se for monocromática, deve-se aplica somente sobre a luminância *Y*.
- O conceito de formas de onda ortogonais é o mesmo conceito utilizado para definir uma base para um espaço R^n, como no caso do R^2 ou R^3. No espaço R^3, por exemplo, define-se 3 vetores (1,0,0), (0,1,0) e (0,0,1) que são a base do sistema. Esses vetores são *LI* (Linearmente independentes), ou seja, um não pode ser expresso como combinação linear dos demais. O mesmo vale para as formas e onda ortogonais. Por exemplo, o ponto (2, 3, 0) = 2 * (1,0,0) + 3 * (0,1,0) + 0 * (0,0,1).

Formas de onda ortogonais



] za
seguinte algoritmo. Observe que forma retirados os coeficientes c(w). Como exercício, implemente este código na Canvas2D.

```
for(u=0; u<8; u++)
   for(v=0; v<8; v++)
      for(x=0; x<8; x++)
         for(y=0; y<8; y++)
         {
            //pix vale entre -1 e 1.
            pix = cos(((2*x+1)*pi*u)/16.0) * cos(((2*y+1)*pi*v)/16.0);
            //normaliza entre 0 e 2
            cor = pix+1;
            //normaliza entre 0 e 255
            cor = cor*127;
            putpixel(u*10 + x, v*10+y, cor);
         }
```

Transformada Cosseno Direta e Inversa

A formulação geral da transformada direta cosseno bidimensional direta (*Forward DCT*) e inversa (*Inverse DCT*), para o caso 2D (imagem) é dada por [8]

$$F(u,v) = \left(\frac{2}{N}\right)^{\frac{1}{2}}\left(\frac{2}{M}\right)^{\frac{1}{2}}\sum_{x=0}^{N-1}\sum_{y=0}^{M-1} f(x,y) * cos\left(\frac{(2x+1)u\pi}{2N}\right) * cos\left(\frac{(2y+1)v\pi}{2M}\right) * C(u) * C(v)$$

$$f(x,y) = \left(\frac{2}{N}\right)^{\frac{1}{2}}\left(\frac{2}{M}\right)^{\frac{1}{2}}\sum_{u=0}^{N-1}\sum_{v=0}^{M-1} F(u,v) * cos\left(\frac{(2x+1)u\pi}{2N}\right) * cos\left(\frac{(2y+1)v\pi}{2M}\right) * C(u) * C(v)$$

onde

$$C(w) = \frac{1}{\sqrt{2}} \; para \; w = 0$$
$$C(w) = 1 \; para \; w = 1,\ldots,N-1$$

- M, N representam a dimensão da imagem (NxM)
- *f(x,y)* representa o valor dos pixels da imagem na posição *(x,y)* → *z = f(x,y)*.
- *F(u,v)* são os coeficientes da FDCT (valores reais associados à frequência da imagem), ou seja, valores que quando multiplicados pelas funções ortogonais vão recriar os dados originais (pixels).
- Cada elemento transformado *F(u,v)* é o produto interno entre *f(x,y)* e NxM vetores básicos
- $-1 \leq \cos(x) \leq 1$
- Aplicando-se sobre uma imagem (ou outra função qualquer) a FDCT e em seguida a IDCT, obtém-se o sinal original. Deve-se observar que podem ser induzidos erros de arredondamento, visto que a matriz *F(u,v)* contém valores reais.

Para o caso unidimensional temos o seguinte

$$F(u) = \left(\frac{2}{M}\right)^{\frac{1}{2}}\sum_{x=0}^{M-1} f(x) * cos\left(\frac{(2x+1)u\pi}{2M}\right) * C(u)$$

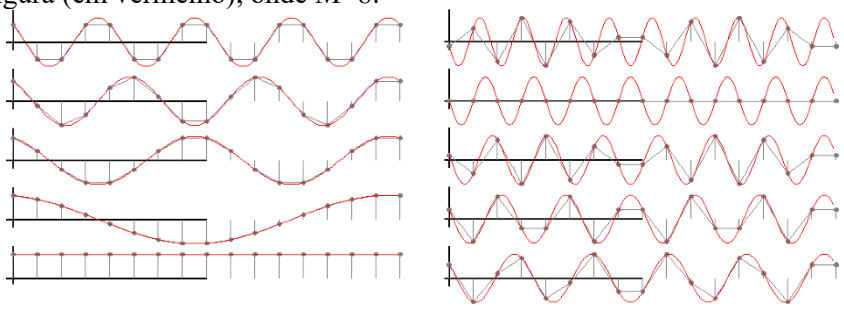
$$f(x) = \left(\frac{2}{M}\right)^{\frac{1}{2}}\sum_{u=0}^{M-1} F(u) * cos\left(\frac{(2x+1)u\pi}{2M}\right) * C(u)$$

Utilizando-se

$$F(u) = cos\left(\frac{(2x+1)u\pi}{2M}\right)$$

pode-se gerar as formas de onda ortogonais para o caso 1D (Figura anterior e figura seguinte). Ou seja, para cada $u = 0, 1, 2, \ldots, M-1$, deve-se iterar em $x = 0,1,2, \ldots, M-1$. Tanto *u* como *x* podem ser extrapolados

para valores ≥ M. O uso de um *x* contínuo gera uma forma de onda completamente diferente, como pode ser visto na seguinte figura (em vermelho), onde M=8.



Essa transformada pode ser expandida para qualquer dimensão. Para o caso de uma matriz 8x8, a transformação direta e inversa é dada por [1]

$$F(u,v) = \frac{1}{4}\left[\sum_{x=0}^{7}\sum_{y=0}^{7} C(u)C(v) * f(x,y) * \cos\frac{(2x+1)u\pi}{16} * \cos\frac{(2y+1)v\pi}{16}\right]$$

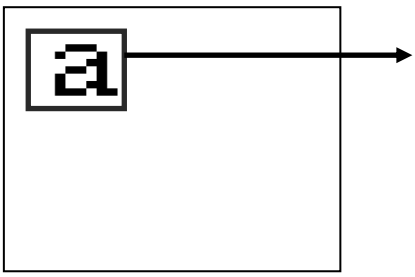
$$f(x,y) = \frac{1}{4}\left[\sum_{u=0}^{7}\sum_{v=0}^{7} C(u)C(v) * F(u,v) * \cos\frac{(2x+1)u\pi}{16} * \cos\frac{(2y+1)v\pi}{16}\right]$$

A transformada direta pode ser otimizada evidenciando-se os fatores *c(w)*

$$F(u,v) = \frac{1}{4}C(u)C(v)\left[\sum_{x=0}^{7}\sum_{y=0}^{7} f(x,y) * \cos\frac{(2x+1)u\pi}{16} * \cos\frac{(2y+1)v\pi}{16}\right]$$

Transformada Cosseno – Matriz de coeficientes

- Imagem com alta variação de frequência (cor) – **Texto.** O valor dos coeficientes é distribuído de forma bastante homogênea na matriz.



F(0,0) →

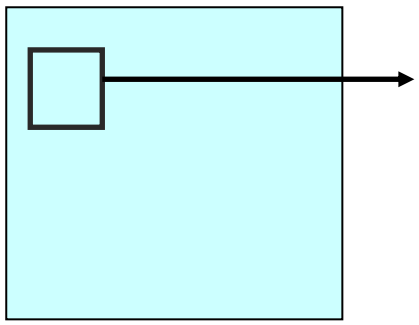
1402	-37	318	-106	255	158	-62	187
229	-1	6	62	-216	-41	-41	-140
-34	-46	-45	120	14	-73	146	-69
162	-17	-51	15	-69	5	-46	3
0	69	34	67	0	-216	-83	-23
162	86	-134	-275	-24	265	69	-17
82	-12	-236	-34	201	6	45	0
0	-39	-33	41	67	-27	0	0

- Imagem com baixa variação de frequência (Cor) – **Pôr do sol.** Os coeficientes com valores significativos se concentram em baixas frequências.



1442	33	41	45	44	38	25	12
29	-4	-7	-1	-1	-2	0	1
49	-3	-1	-5	-2	-2	-2	0
43	-2	0	-2	-2	-2	-2	-2
45	-3	-3	-3	-3	-1	0	-2
37	-4	-1	-2	-2	-1	-2	0
26	0	0	-1	-2	0	0	0
13	0	-2	0	-3	-2	0	0

- Imagem sem variação de frequência (Cor). Apenas o coeficiente DC tem valor não zero, visto que a imagem somente possui frequência zero.



2040	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0

FDCT em Java

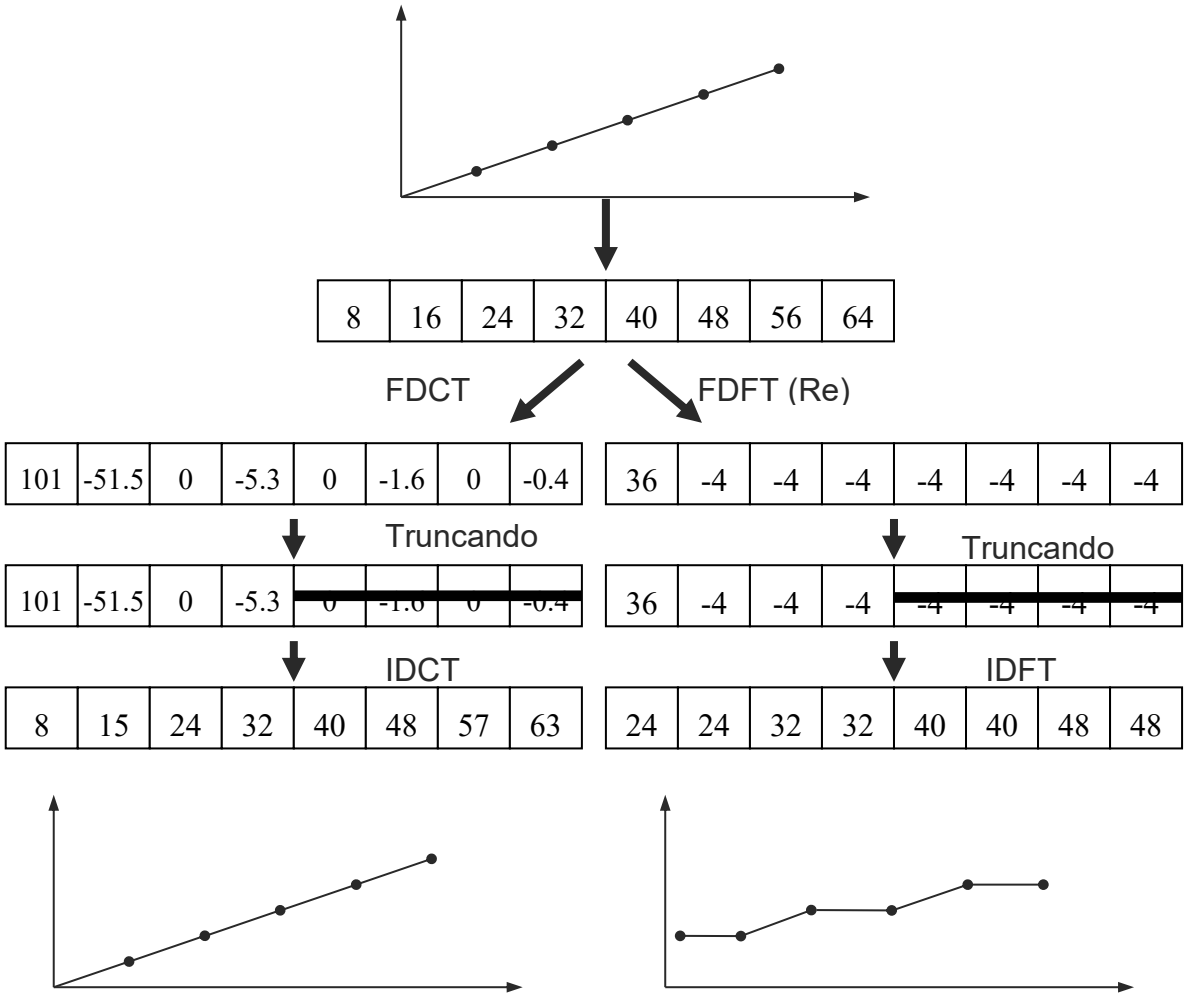
```
Matriz fdct(Imagem i) //forward discrete cosine transform
{
   int u, v, x, y;
   double pix, pi = Math.PI;
   for(u=0; u<8; u++)
   {
      for(v=0; v<8; v++)
      {
         pix = 0.0;
         for(x=0; x<8; x++)
         {
            for(y=0; y<8; y++)
            {
               pix += i.mat[x][y] *
                      cos(((2.0*x+1.0)*pi*u)/16.0) *
                      cos(((2.0*y+1.0)*pi*v)/16.0);
            }
         }
         m.mat[u][v] = (pix/4.0)*C(u)*C(v);
      }
   }
   return m;
}
```

IDCT em Java

```
Imagem idct(Matriz m) //inverse discrete cosine transform
{
   int u, v, x, y;
   double pix, pi = Math.PI;
   for(x=0; x<8; x++)
   {
      for(y=0; y<8; y++)
      {
         pix = 0.0;
         for(u=0; u<8; u++)
         {
            for(v=0; v<8; v++)
            {
               pix += C(u)*C(v)*m.mat[u][v] *
                      cos(((2.0*x+1)*pi*u)/16.0) *
                      cos(((2.0*y+1)*pi*v)/16.0);
            }
         }
         i.mat[x][y] = (int)Math.round(pix/4.0);
      }
   }
   return i;
}
```

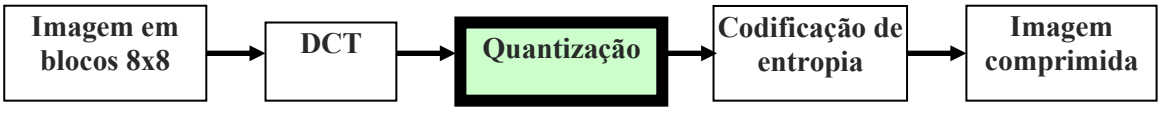
DFT x DCT [10]

Neste exemplo faz-se um comparativo da aplicação da DCT e da DFT (Discrete Fourier Transform). Para obter compressão da imagem, deve-se remover frequências altas. Pode-se observar que, para o mesmo conjunto de entrada, a DCT concentra a informação em coeficientes maiores (baixas frequências). Desta forma, coeficientes de alta frequência possuem baixa relevância na imagem e ao serem eliminados, não causam muito erro na reconstrução da imagem. A transformada de Fourier, para esta aplicação não se mostra tão eficiente, pois distribui de forma muito homogênea os coeficientes em todo espectro de frequência. São apresentados apenas as componentes Real da DFT.



Este exemplo pode ser utilizado para testar se a implementação da FDCT e a IDCT para o caso 1D com 8 amostras está correta.

Compressão JPEG - Quantização



- Obtém compressão pela representação dos coeficientes *F(u,v)* com menor precisão (Nº de bits).
- Consiste em dividir cada componente gerado pela DCT por um valor inteiro, definida em uma matriz de quantização (com valores entre 1 e 255). A escolha destes valores é difícil [1, 13].
- Objetivo de descartar informação com pouco valor. É a principal fonte de perda de informação.

$$F^{Q}(u,v) = ArredondamentoInteiro\left(\frac{F(u,v)}{Q(u,v)}\right)$$

$$F^{Q'} = F^{Q}(u,v) * Q(u,v)$$ (processo inverso usado na descompressão da imagem)

- Q[i, j] = 1 + (1 + i + j) * FatorQuantização
- FatorQuantização é considerado entre 2 e 25 e os índices iniciam no zero. Quanto maior for, maiores serão as perdas. Dessa maneira é feita uma quantização por zona, sendo os coeficientes associados às mais altas frequências quantizados com mais severidade.

Matriz de quantização para fator 2

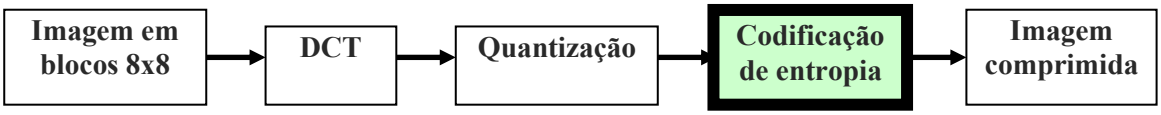
3	5	7	9	11	13	15	17
5	7	9	11	13	15	17	19
7	9	11	13	15	17	19	21
9	11	13	15	17	19	21	23
11	13	15	17	19	21	23	25
13	15	17	19	21	23	25	27
15	17	19	21	23	25	27	29
17	19	21	23	25	27	29	31

Wallance [1]

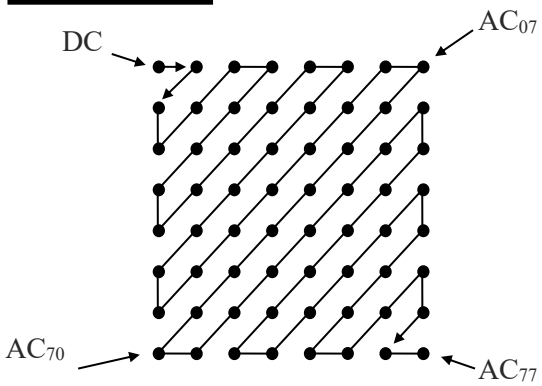
16	11	10	16	24	40	51	61
12	12	14	19	26	58	60	55
14	13	16	24	40	57	69	56
14	17	22	29	51	87	80	62
18	22	37	56	68	109	103	77
24	35	55	64	81	104	113	92
49	64	78	87	103	121	120	101
72	92	95	98	112	100	103	99

Recomenda-se [13] como leitura adicional.

Compressão JPEG - Entropia



- Após a quantização, tem-se uma matriz de coeficientes de tamanho 8x8, onde o coeficiente (0,0) é chamado de DC, e os demais AC. O coeficiente DC mede a média de todos os 64 coeficientes, e por isso é o valor mais significativo da matriz [1].
- O zig-zag é usado para concentrar coeficientes de baixa frequência antes dos de alta, facilitando assim a codificação de entropia. A matriz 8x8 é transformada em um vetor de 64 posições.
- Métodos de entropia: aplica-se *Huffman coding* ou *Arithmetic coding* sobre o vetor de 64 posições para fazer a compactação.



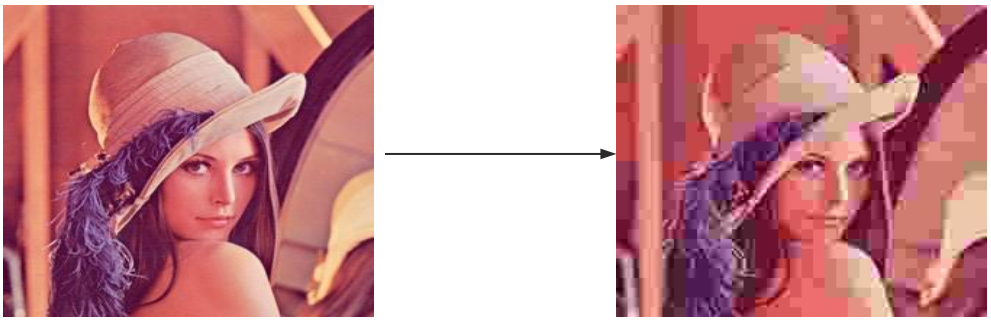
Compressão JPEG – Modo Progressivo

- Cada componente da imagem é codificado em múltiplas passadas.
- Permite em um ambiente de rede, por exemplo, que a imagem seja codificada de modo gradativo (múltiplos scans). No primeiro scan tem-se uma versão grosseira da imagem. Nos subsequentes, versões mais detalhadas.
- Faz uso de duas abordagens:
  - *Spectral selection*: Codifica os coeficientes segundo a ordem do zig-zag (alta ou baixa freqüências)

- *Sucessive aproximation*: codifica a cada passada, inicialmente os bits mais significativos até os menos significativos.

Deve-se observar que os dados salvos no arquivo JPEG representam frequências e não cores. Quando o arquivo é carregado, todo o processo é executado na ordem inversa, e após a realização da IDCT, têm-se novamente cores.

Lena". Exemplo de uma imagem normal e a mesma após processo de compressão muito elevado. [4]



Sumário da compressão JPEG

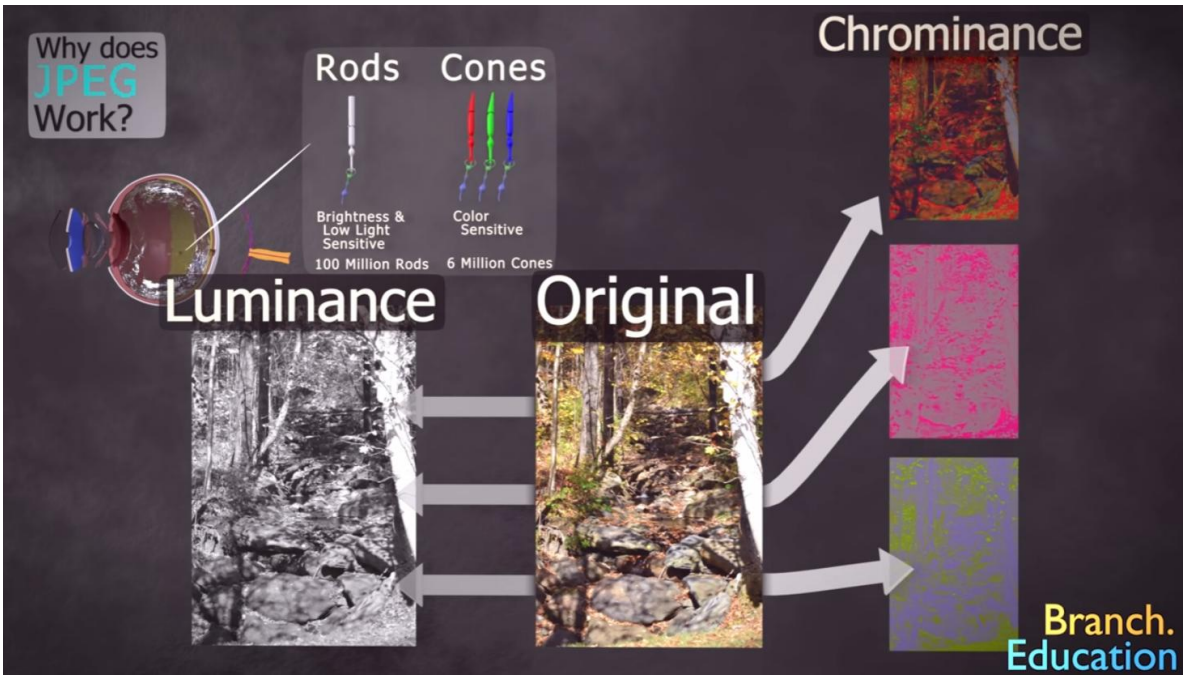
Para sumarizar todo processo, vamos considerar que a entrada seja uma imagem grayscale (uma banda de cor apenas) de 4x4 pixels.

1. Entrada: matriz com 4x4 inteiros de 8 bits (unsigned char da linguagem C ou byte do C#).
2. Após a aplicar a DCT, tem-se uma matriz de coeficientes de frequência de 4x4 floats. Observe que o espaço de armazenamento aumentou 4x. Cada elemento dessa matriz representa um coeficiente associado uma frequência específica.
3. Na quantização, utiliza-se uma matriz de 4x4 inteiros de 8 bits. A matriz de coeficientes é dividida pela matriz de quantização (divisão inteira). O elemento coef[i][j] é dividido por quantiza[i][j]. Após esse processo, tem-se uma matriz 4x4 de inteiros de 32 bits, sendo a maioria dos valores próximos ou igual de zero, exceto o coeficiente DC (mat[0][0]), que possui um valor bem elevado. Nessa etapa ocorreu a compressão (perda de informação), pela divisão inteira. Quanto maior forem os valores da matriz de quantização, maior será a compressão.
4. Na entropia os valores da matriz mat da etapa anterior são reorganizados em zig-zag de forma que os valores menores fiquem próximos uns dos outros no vetor resultante. Sobre esse vetor são aplicados algoritmos de compactação (sem perda).
5. O vetor compactado é salvo em disco. Observe que o que é salvo em disco são coeficientes de frequência, e não cores.
6. Para carregar uma imagem JPEG, executa-se o processo na ordem inversa. O bloco (vetor de dados) é lido do disco, aplica-se o algoritmo de descompactação, aplica-se o zig-zag na ordem contrária, gerando-se novamente uma matriz de dados, multiplica-se pela **mesma** matriz de quantização, aplica-se a IDCT, e obtêm-se o bloco de 4x4 pixels, que geralmente não será idêntico ao bloco original.

Para uma explicação visual animada de todo processo de compressão JPEG, veja o seguinte vídeo, do canal Branch Education:

How are Images Compressed? [46MB ↘↘ 4.07MB] JPEG In Depth

https://www.youtube.com/watch?v=Kv1Hiv3ox8I&list=PL6rx9p3tbsMsZ9hUvU-kDOXc8Fot04Hhu&index=4



Programa DCT [8]

- Ilustra o processo da transformada co-seno direta e inversa
- Mostra as funções básicas
- Mostra o valor dos pixels (componente R) do primeiro bloco 8x8 da imagem e seus respectivos coeficientes da DCT
- Mostra as componentes R, G, B e Y da imagem.

Imagem Original

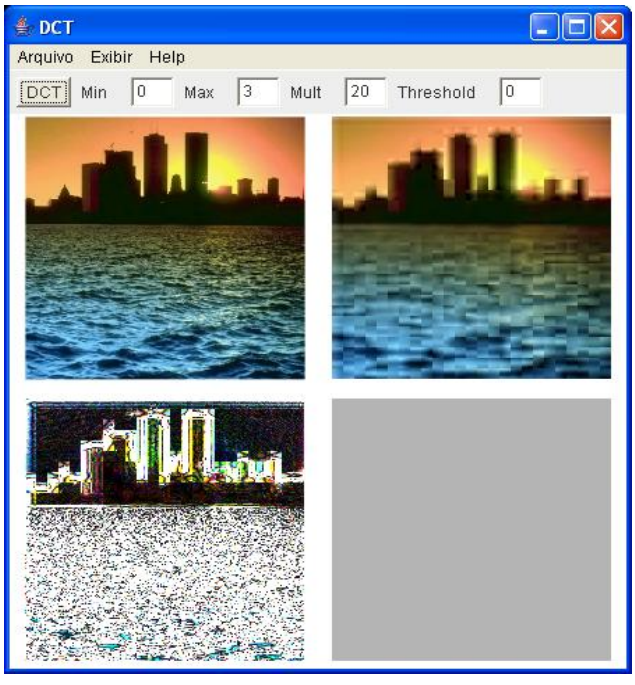
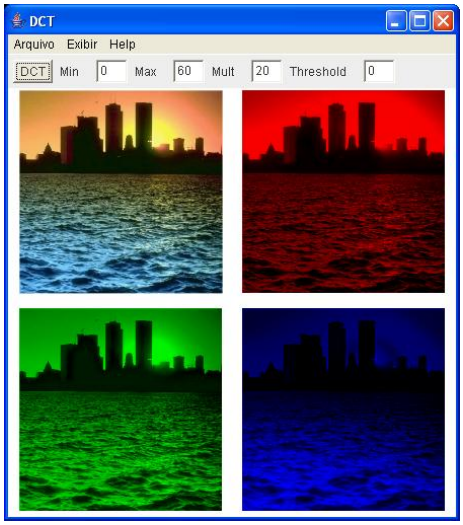
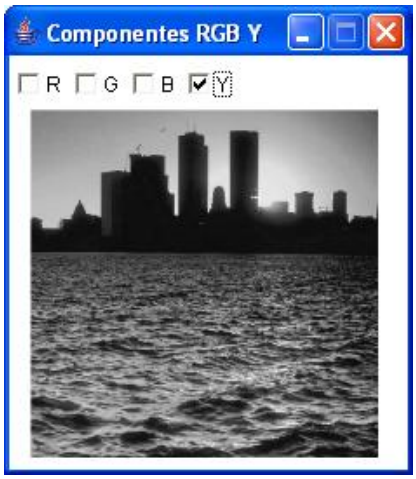


Imagem Reconstruída

Diferença entre original e reconstruída

Taxa de compressão



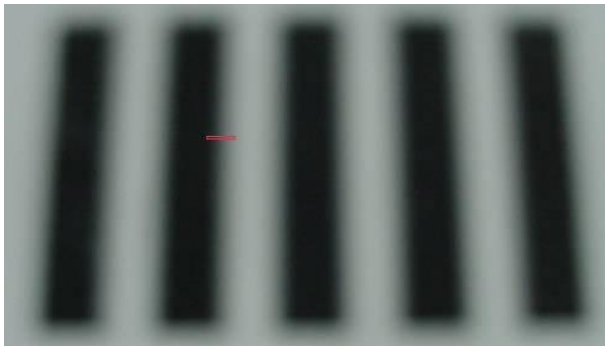


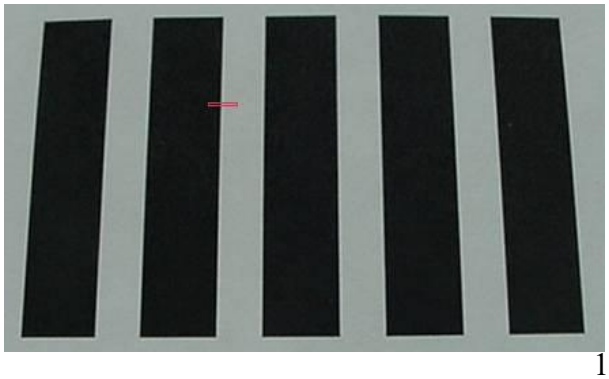




Aplicação de Análise de Frequências [12]

> ***Passive*** *autofocus, commonly found on single-lens reflex (SLR) autofocus cameras, determines the distance to the subject by* ***computer analysis*** *of the image itself. The camera actually looks at the scene and drives the lens back and forth searching for the best focus. A typical autofocus sensor is a* ***charge-coupled device*** *(CCD) that provides input to algorithms that compute the contrast of the actual picture elements. The CCD is typically a single strip of 100 or 200 pixels. Light from the scene hits this strip and the microprocessor looks at the values from each pixel. The following images help you understand what the camera sees.* **Resumindo: a imagem está em foco quando existirem altas frequências, como mostrado na segunda imagem.**





Tópicos adicionais

Funções ortogonais

Duas funções são **ortogonais** quando o seu produto interno (produto escalar) for zero, ou seja, formam um ângulo de 90 graus entre si [14]. Em termos matemáticos, dadas duas funções f(x) e g(x), temos que o produto entre elas tem que ser zero.

$$\int_{-\infty}^{\infty} f(x) \cdot g(x) = 0$$

Para funções trigonométricas, pode-se definir o intervalo de integração como múltiplo de π.

$$\int_{0}^{\pi} \sin(x) \cdot \cos(x) = 0$$

Isso pode ser verificado utilizando-se o seguinte algoritmo

```
#include<stdio.h>
#include<math.h>

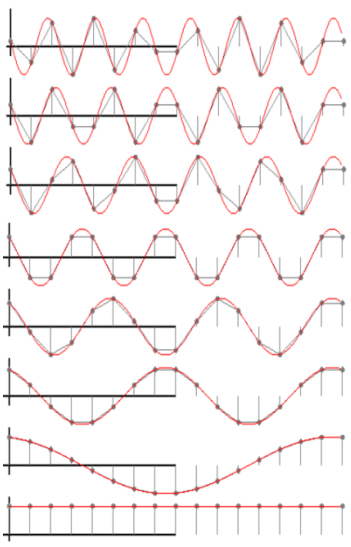
void main()
{
   double soma = 0;
   double ang = 0;
   for(ang=0; ang < M_PI; ang+=0.00001)
   {
       soma = soma + sin(ang)*cos(ang);
   }
   printf("%lf", soma);
}
```

Considerando-se a transformada DCT 1D com 8 amostras, temos a seguinte formulação:

$$F(u) = \frac{1}{2} C(u) \sum_{x=0}^{7} f(x) * \cos \frac{(2x+1)u\pi}{16}$$

Para mostrar que essas 8 funções são ortogonais, utiliza-se o seguinte algoritmo. Considerando-se quaisquer dois $u_1 \neq u_2$, temos que o produto interno das funções sempre dá como resultado valor zero.

```
void ortogonal()
{
    double u1, u2, x, f1, f2;
    double soma = 0;
    for(x=0; x<8; x++)
    {
        u1 = 3;
        u2 = 2;
        f1 = cos(((2*x+1)*PI*u1)/16.0);
        f2 = cos(((2*x+1)*PI*u2)/16.0);
        soma += f1*f2;
    }
    printf("%lf", soma);
}
```



Exercícios

1. Faça um programa para carregar uma imagem colorida em formato BMP (usar funções do demo 15_GL_texture). Após, implemente funções para:
    a. Gerar o histograma de cada canal
    b. Gerar imagem com cada canal de cor separado
    c. Converter para imagem monocromática (tons de cinza)
    d. Fazer rotação na imagem em 90 graus
    e. Fazer reescala da imagem
    f. Aumentar e diminuir o brilho da imagem.
    g. Aplicar a DCT e fazer a redução de altas frequências
    h. Salvar a nova imagem em disco.

Referências:

[1] Wallace, G. K., *The JPEG Still Picture Compression Standard*, IEEE Transactions on Consumer Electronics, 1991

[2] Soares, L. F. G. *Notas de aula, Curso de Fundamentos de Multimídia*, PUC-Rio.

[3] Pozzer, C. T. *Programa JPEG Compressor*. Disponível em: http://www-usr.inf.ufsm.br/~pozzer/, na seção Trabalhos.

[4] Rigotti, H. G. *Codificação de Imagem Usando Transformada Cosseno Discreta*. Dissertação de Mestrado, UFMS, 2004.

[5] Marshall, D. *DCT*. Disponível em: http://www.cs.cf.ac.uk/Dave/Multimedia/node231.html

[6] DCT – Disponível em http://en.wikipedia.org/wiki/Discrete_cosine_transform

[7] Guetzli: Perceptually Guided JPEG Encoder

[8] Funções ortogonais. https://en.wikipedia.org/wiki/Orthogonal_functions